DevBlog – Node.js, MongoDB & Docker Project

This project is a **DevBlog backend application** built with **Node.js, Express, MongoDB**, and **Docker**.
It includes user management, admin authentication, MongoDB integration, and containerized deployment.

The goal of this project was to:

- Set up a Node.js backend
- Connect MongoDB locally and inside Docker
- Create users via a CLI tool
- Implement admin authentication
- Debug real-world Docker and environment issues
- Learn container networking and session handling

Tech Stack

- **Node.js** (v19)
- **Express.js**
- **MongoDB**
- **Mongoose**
- **EJS** (views)
- **Docker & Docker Compose**
- **MongoDB Compass / Mongo Express**
- **express-session**
- **dotenv**

Project Structure

```
devblog/
├── app.js
├── bin/
│ └── www
├── routes/
│ ├── index.js
│ └── admin.js
├── models/
│ └── User.js
├── utils/
│ └── db.js
├── views/
│ ├── admin/
│ └── error.ejs
├── public/
├── create-user-cli.js
├── Dockerfile
├── docker-compose.yml
```

```
├── .dockerignore
├── .env
├── package.json
└── README.md
```

#Environment Variables

Create a `.env` file in the project root:

```
.env
MONGO_URI=mongodb://127.0.0.1:27017/devblog npm start
```

Running the App Locally (Without Docker)

Install dependencies

```
nvm install 19
node -v
npm -v
nvm use 19
```

Start MongoDB (Windows)

After installing MongoDB, you start the service

```
Get-Service MongoDB
Start-Service MongoDB
```

Creating Users via CLI

A CLI tool was created to insert users directly into MongoDB.

```
Run the CLI
node create-user-cli.js
```

Docker Setup

Dockerfile (Node + Alpine)

The app is containerized using a lightweight Node Alpine image. I wrote a Dockerfile from the project root directory.

Dockerfile

```
FROM node:19.9.0-alpine

WORKDIR /app

COPY package.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD [ "npm", "start" ]
```

Build the image

```
docker build -t devblog .
```

Run container

```
docker run --rm --name devblog -p 3000:3000 devblog:latest

or

docker run -p 3000:3000 --env-file .env devblog
```

Docker Compose

Docker Compose is used to manage the application container.

Build and start

```
docker compose up --build

docker ps

Stop services
docker compose down --remove-orphans
```

```
droyal@DESKTOP-DARYL MINGW64 ~/Documents/DevOps Mentorship/week2-Nodejs/devblog (main)
$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED        STATUS        PORTS
       NAMES
812aa7a014a7   devblog-web     "docker-entrypoint.s…"   2 hours ago    Up 2 hours    0.0.0.0:3000->3000/tcp, [::]:3000->300
0/tcp   devblog-web-1
e61326ec19a8   mongo-express   "/sbin/tini -- /dock…"   19 hours ago   Up 19 hours   0.0.0.0:8081->8081/tcp, [::]:8081->808
1/tcp   mongo-express

droyal@DESKTOP-DARYL MINGW64 ~/Documents/DevOps Mentorship/week2-Nodejs/devblog (main)
$
```

To access or navigate into the container

```
docker exec -it devblog-web-1 sh
```

I noticed that alpine linux does not include `bash` by default it uses `sh`to naviagte into the container.

`error from docker exec bash`

```
droyal@DESKTOP-DARYL MINGW64 ~/Documents/DevOps Mentorship/week2-Nodejs/devblog (main)
$ docker exec -it devblog-web-1 bash
OCI runtime exec failed: exec failed: unable to start container process: exec: "bash": executable file not found in $PATH

droyal@DESKTOP-DARYL MINGW64 ~/Documents/DevOps Mentorship/week2-Nodejs/devblog (main)
$
```

access with `sh`

```
droyal@DESKTOP-DARYL MINGW64 ~/Documents/DevOps Mentorship/week2-Nodejs/devblog (main)
$ docker exec -it devblog-web-1 sh
/app # ls
Dockerfile          app.js              docker-compose.yml  node_modules        public              utils
LICENSE             bin                 middlewares         package-lock.json   routes              views
README.md           create-user-cli.js  models              package.json        tailwind.config.js
/app #
```

MongoDB & Mongo Express

MongoDB Runs locally on Windows

```
Port: 27017
```

Accessed via Mongoose

MongoDB Compass

Used to visually inspect databases and collections.

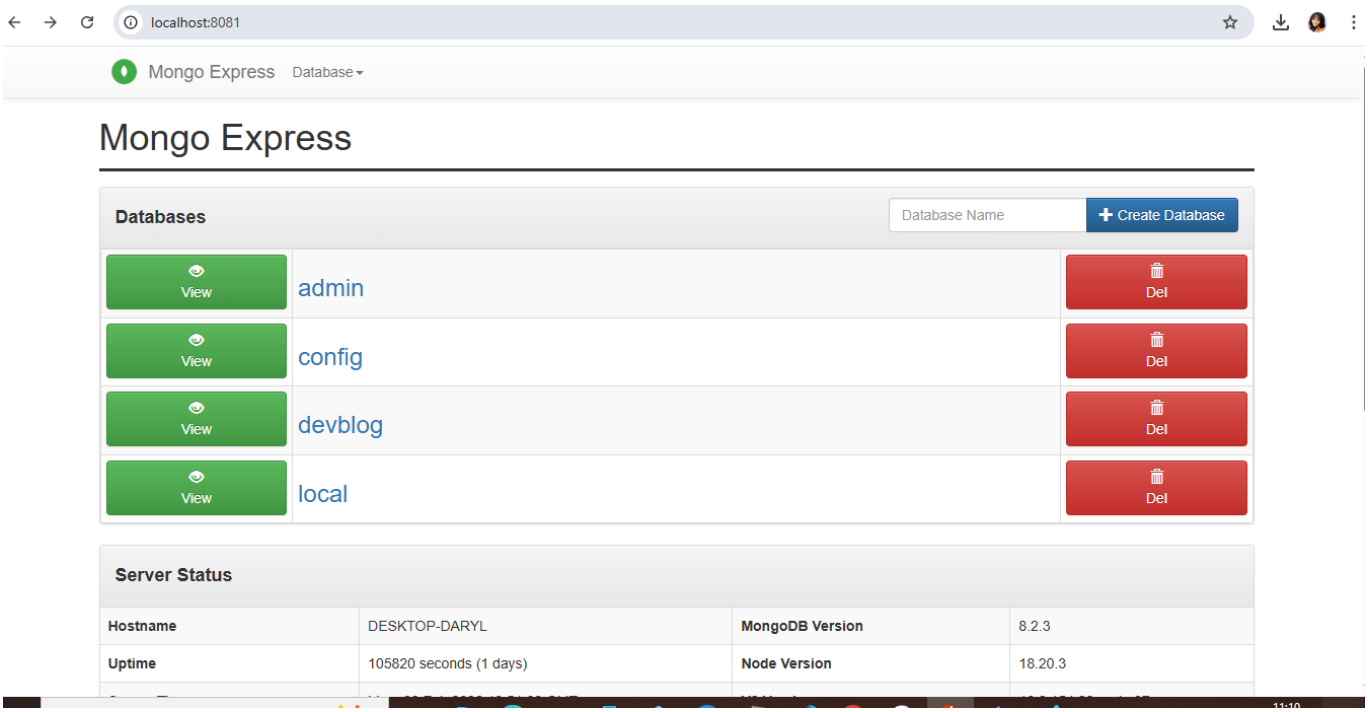
Mongo Express

Run Mongo Express on my terminal to view MongoDB in the browser:

```
docker run -d \
  --name mongo-express \
  -p 8081:8081 \
  -e ME_CONFIG_MONGODB_SERVER=host.docker.internal \
  -e ME_CONFIG_BASICAUTH=false \
  mongo-express
```

Access:

```
http://localhost:8081
```



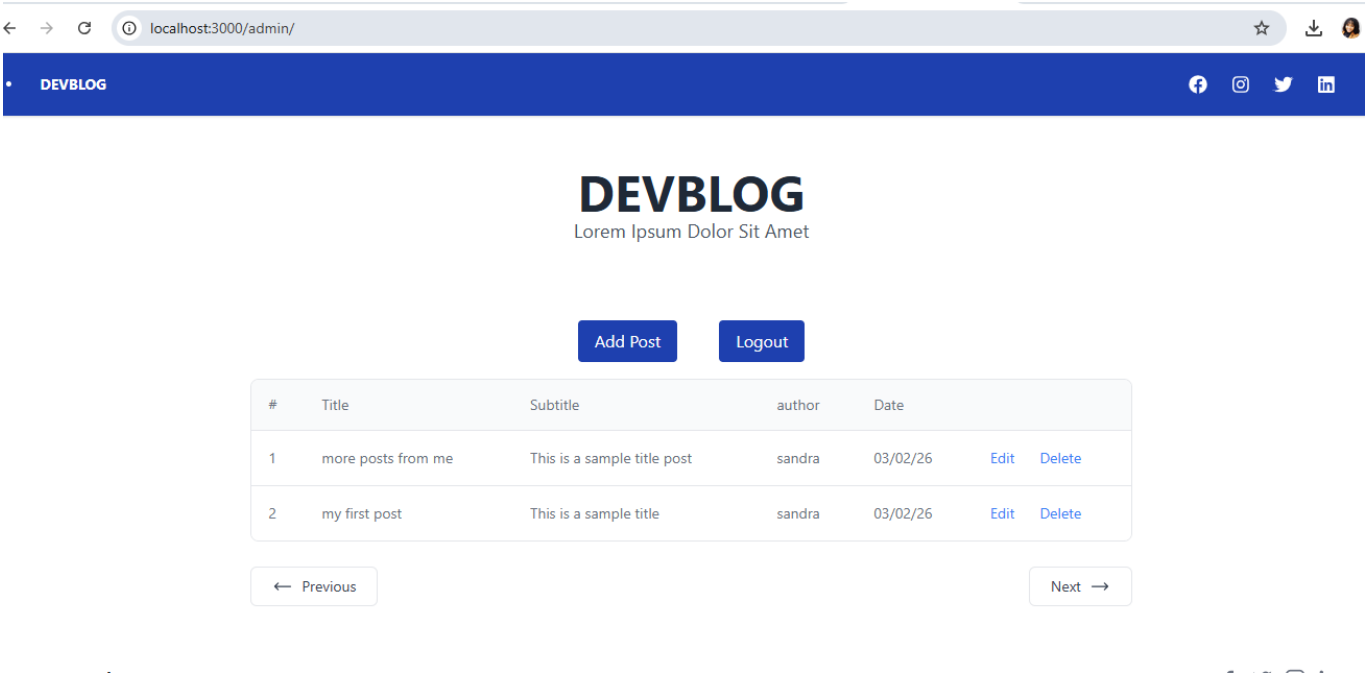
Admin Authentication Sessions

Admin authentication uses express-session. meanwhile I was finding it difficult to login as an admin. then I added this line in the `app.js file`

Installed with:

```
npm install express-session --save

Configured in app.js:

app.use(session({
  secret: "devblog-secret-key",
  resave: false,
  saveUninitialized: false
}));
```

- Admin access

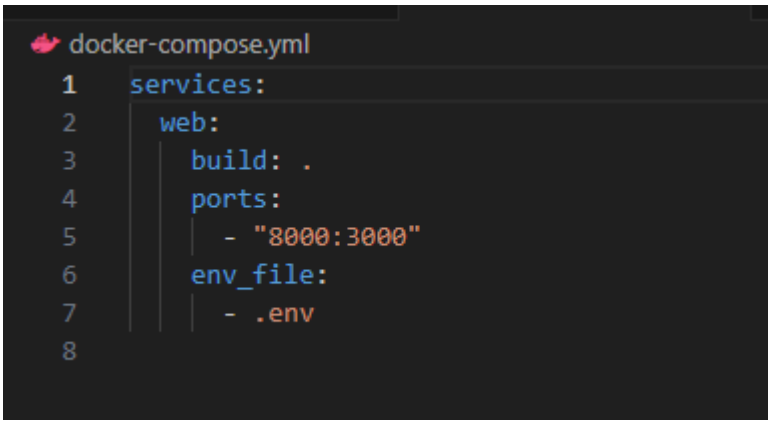


Routes

```
Admin login:

/login


Admin dashboard (protected):

/admin


Middleware ensures only authenticated users can access admin routes.
```

NEXT: docker-compose.yml

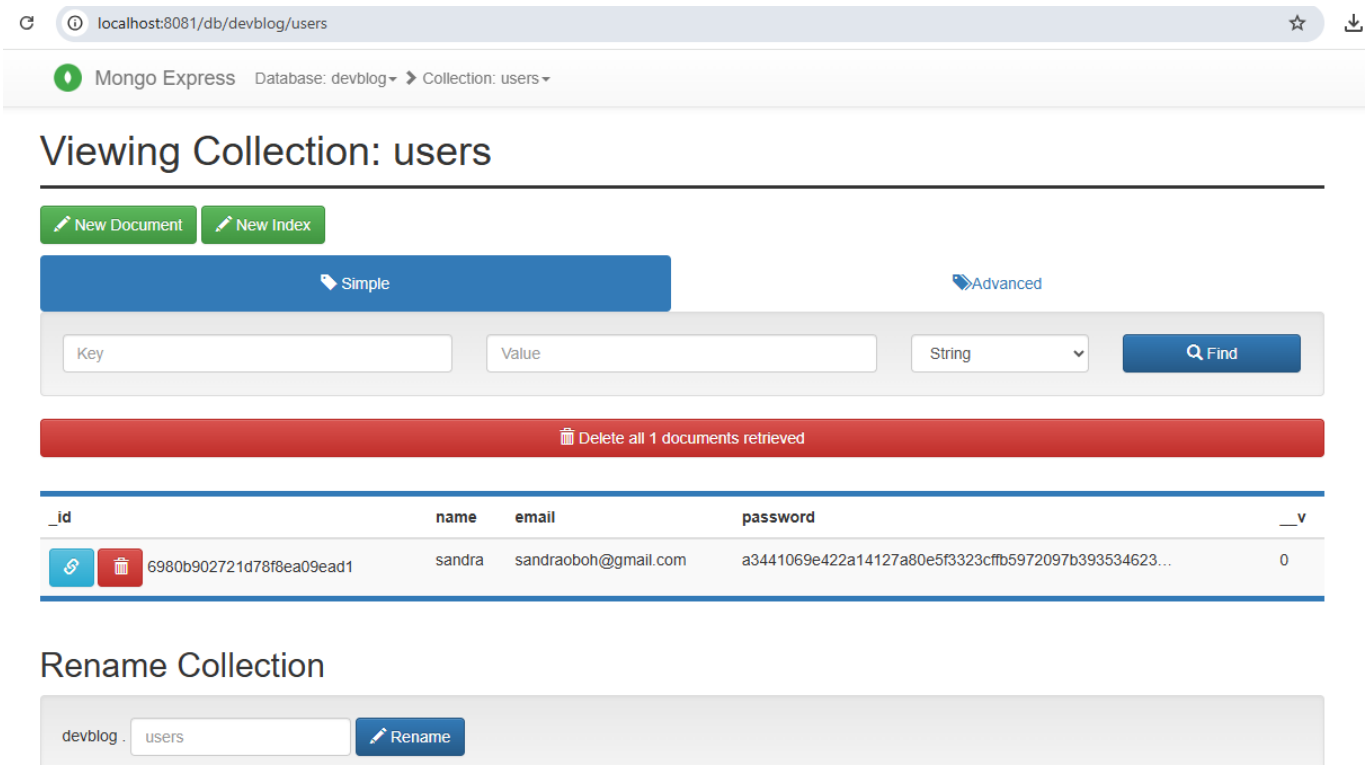
I created my yml file



- ran `docker compose up`

created a user cli

- `node create-user-cli.js`



NEXT START.SH Project